AI335L — Deep Learning Lab

Phase 1 Project Roadmap

MultiShield

A Unified Multimodal Deep Learning System for Cybersecurity Threat Detection

Detecting phishing, fake news, deepfakes, and behavioral anomalies through intelligent fusion of CNN, Transformer, LSTM, and generative models.

Field	Details
Project Title	MultiShield — Multimodal Threat Detection System
Course Code	AI335L — Deep Learning Lab
Instructor	Sir Haseeb
Semester	Spring 2026
Submission Date	April 27, 2026

Reg. ID	Name	Role
S2024CS020	Hanzala Ahmed (Team Lead)	Project Lead / Transformer Module
S2024CS001	Manahil Fatima	CNN & LSTM Module
S2024CS027	Uzair Ahmad	Documentation

Section 2: Executive Summary / Abstract

The Problem: Every day, people and organizations are targeted by online threats — fake emails trying to steal passwords, AI-generated fake videos, false news stories, and employees misusing their access to company systems. These attacks are growing faster and smarter every year.

Why It Matters: Most existing tools only fight one type of threat at a time. A separate tool for phishing, another for deepfakes, another for fake news — none of them talk to each other. Attackers take advantage of these blind spots by combining multiple attack types at once, slipping through the cracks.

Proposed Approach: MultiShield tackles all of these threats together in one system. It looks at text, images, and user behavior at the same time, combines what it learns from each, and makes one unified decision — safe or threat.

Deep Learning Components: The system uses five AI models working together:

- A **CNN** to spot fake or manipulated images
- A **BERT Transformer** to detect phishing emails and fake news articles
- An **LSTM** to catch suspicious patterns in how users behave over time
- A **GAN and Autoencoder** to detect unusual activity it has never seen before
- A **fusion MLP** that takes all the above findings and makes the final call

Data Sources: The system is trained on four real-world, publicly available datasets covering fake news articles, phishing emails, deepfake videos, and insider threat logs from actual organizations.

Expected Outcome: The end result will be a working web app where anyone can submit text, an image, or behavioral data and instantly get a threat assessment — with over 90% accuracy across all threat types.

Section 3: Problem Statement & Motivation

3.1 The Scale of the Problem

Cybersecurity threats have reached epidemic proportions in the digital age. According to the Anti-Phishing Working Group (APWG), phishing attacks exceeded 1.2 million unique incidents in 2023 alone, causing estimated global losses of over USD 52 billion. Simultaneously, the proliferation of AI-generated deepfake media has increased by over 900% since 2019 (Deeptrace Labs, 2023), with synthetic images and videos being weaponized for fraud, non-consensual exploitation, and political disinformation. Fake news, amplified through social media algorithms, was identified as a primary driver of societal polarization in studies covering 46 countries (Reuters Institute Digital News Report, 2023). Insider threats and anomalous behavioral patterns account for 34% of all data breaches (Verizon DBIR, 2023), costing organizations an average of USD 4.45 million per incident (IBM Cost of Data Breach Report, 2023).

3.2 Who Is Affected

The impact spans individuals, enterprises, governments, and democratic institutions. Individual users fall victim to phishing scams and identity theft. Enterprises suffer reputational damage and financial losses from deepfake fraud and insider attacks. Governments and electoral bodies are increasingly targeted by coordinated disinformation campaigns combining fake news and synthetic media. Healthcare providers face ransomware delivered via phishing. Financial institutions experience social engineering attacks combining multiple threat vectors simultaneously.

3.3 Limitations of Existing Solutions

Current commercial and academic solutions are predominantly single-modality and single-task in design. Email filters use rule-based or shallow ML approaches that fail against novel phishing variants. Deepfake detectors are trained on specific GAN architectures and fail to generalize. Fake news classifiers operate only on textual features, ignoring accompanying manipulated images. Behavioral anomaly detection systems operate in complete isolation from the content layer. This siloed architecture creates critical blind spots:

- No cross-modal correlation: a phishing email containing a deepfake image is analyzed separately, missing combined signals.
- High false positive rates: single-modality models lack contextual grounding, flagging legitimate content incorrectly.
- Poor generalization: models trained on one threat type fail catastrophically when adversaries combine vectors.
- Operational overhead: organizations must maintain, update, and integrate multiple disparate tools.
- No unified risk score: security teams receive fragmented signals rather than a single, actionable threat assessment.

3.4 The Gap This Project Addresses

There is a critical and commercially significant gap for a unified multimodal deep learning system that simultaneously ingests text, image, and behavioral data, applies specialized neural architectures to each modality, and fuses the resulting embeddings into a single, explainable threat classification. MultiShield addresses this gap by combining state-of-the-art architectures — CNN, Transformer, LSTM,

GAN/Autoencoder — under a single MLP-based fusion classifier, enabling correlated, cross-modal threat intelligence that no single-modality solution can provide. This approach is not achievable with logistic regression or traditional ML: the complexity of image manipulation detection, contextual language understanding, and long-range temporal dependency modeling inherently requires deep learning.

Section 4: Literature Review

4.1 Foundational Papers

[F1] Ismail et al. (2026) — A Comprehensive Review of Convolutional Neural Networks: Foundations, Enhancements and Applications

Problem: Consolidating a decade of CNN advancements across computer vision and NLP. Approach: Systematic analysis of foundational CNN concepts, structural enhancements including advanced activation functions and novel pooling strategies, and applications in image classification, medical imaging, and autonomous systems. This survey contextualizes the EfficientNet-B0 architecture MultiShield uses for deepfake detection within the broader CNN design landscape. Limitation: Survey in nature; does not address adversarial robustness or GAN-generated image detection specifically.

[F2] Sun et al. (2026) — Efficient Attention Mechanisms for Large Language Models: A Survey

Problem: The quadratic time and memory complexity of self-attention remains a fundamental obstacle to efficient long-context modeling in Transformer-based architectures. Approach: Surveyed two principal categories of efficient attention — linear attention methods using kernel approximations and recurrent formulations, and sparse attention mechanisms — providing a unified view of scalable inference strategies. Directly informs MultiShield's BERT fine-tuning strategy and the decision to freeze lower Transformer layers to reduce computational overhead in phishing and fake news classification. Limitation: Survey-level coverage; practical deployment guidance for domain-specific fine-tuning tasks is limited.

4.2 Recent Papers (Within Last 3 Years)

[R1] Jugade et al. (2026) — Deepfake Detection via Spatial-Temporal Deep Networks: Leveraging CNNs and LSTMs for Enhanced Accuracy

Problem: Benchmarking deepfake detection models combining spatial and temporal feature descriptions.

Approach: Systematically compared pretrained CNN architectures — ResNeXt50, XceptionNet, MobileNet, and VGG16 — coupled with LSTM networks, using CNNs as spatial feature extractors while LSTM captures temporal relationships across video frames.

Dataset: FaceForensics++, evaluated with k-fold cross-validation across accuracy, precision, recall, F1-score, and AUC-ROC.

Limitation: Evaluated only known manipulation types; performance degrades significantly on unseen or novel GAN architectures — motivating MultiShield's WGAN-GP discriminator to handle previously unseen deepfake patterns.

[R2] Petmezas et al. (2025) — Video Deepfake Detection Using a Hybrid CNN-LSTM-Transformer Model for Identity Verification

Problem: Detecting manipulated videos with hybrid architectures combining spatial, temporal, and attention-based modeling.

Approach: Combined CNN, LSTM, and Transformer layers into a unified pipeline for video deepfake detection.

Results showed that AUC improves noticeably as the number of layers increases from one to six, suggesting that deeper models capture more complex patterns essential for accurate detection. However, beyond six layers, gains became marginal.

Limitation: High computational overhead limits real-time deployment — reinforcing MultiShield's decision to use static image frames rather than full video streams within its CNN module.

[R3] Raza et al. (2025) — Enhancing Fake News Detection with Transformer-Based Deep Learning: A Multidisciplinary

Problem: Reliable automated detection of misinformation across digital platforms.

Approach: Applied a BERT model enhanced with a progressive training methodology, allowing incremental learning of linguistic nuances differentiating factual from fabricated content.

Dataset: WELFake dataset comprising 72,134 articles.

Results: Achieved 95.3% accuracy, F1-score of 0.953, precision of 0.952, and recall of 0.954.

Limitation: Text-only framework; ignores manipulated images frequently embedded alongside fake news articles — a cross-modal gap that MultiShield directly addresses by fusing CNN visual analysis with BERT classification

[R4] Saadi et al. (2025) — Enhancing Fake News Detection with Transformer Models and Summarization

Problem: Reducing computational cost while improving fake news classification accuracy.

Approach: Evaluated BERT, RoBERTa, and XLNet using supervised and unsupervised techniques, optimizing classification accuracy through text summarization.

Results: RoBERTa fine-tuned with summarized content achieved 98.39% accuracy, outperforming all other models. Additionally assessed AI-generated misinformation using GPT-2, confirming that transformer models effectively distinguish real from synthetic news.

Limitation: Relies purely on textual features; does not account for multimodal misinformation that pairs fabricated text with synthetic imagery.

[R5] Anonymous et al. (2026) — Phishing Email Detection Using BERT and RoBERTa

Problem: Reducing computational cost while improving fake news classification accuracy.

Approach: Evaluated BERT, RoBERTa, and XLNet using supervised and unsupervised techniques, optimizing classification accuracy through text summarization.

Results: RoBERTa fine-tuned with summarized content achieved 98.39% accuracy, outperforming all other models. Additionally assessed AI-generated misinformation using GPT-2, confirming that transformer models effectively distinguish real from synthetic news.

Limitation: Relies purely on textual features; does not account for multimodal misinformation that pairs fabricated text with synthetic imagery.

[R6] Yumlembam et al. (2025) — Insider Threat Detection Using GCN and Bi-LSTM with Explicit and Implicit Graph Representations

Problem: Detecting concealed malicious insider activities by trusted users within organizational networks.

Approach: Integrated explicit graph structures built from predefined activity rules with implicit graphs derived from feature similarities using the Gumbel-softmax trick, combined with a temporal Bi-LSTM component to analyze user behavior sequences.

Results demonstrated robust detection, distinguishing normal from abnormal activities even in complex scenarios.

Limitation: Graph construction from raw logs adds significant preprocessing overhead; MultiShield simplifies this by applying a flat BiLSTM directly on event sequences from the CERT Insider Threat Dataset.

4.3 Additional Sources

[A1] Benabderrahmane et al. (2025) — Attention Adversarial Dual AutoEncoder for Cybersecurity Anomaly Detection

Problem: Detecting Advanced Persistent Threats where labeled data is extremely scarce in real-world cybersecurity environments.

Approach: Proposed an AutoEncoder-based unsupervised anomaly detection framework augmented with active learning, selectively querying an oracle for uncertain samples to reduce labeling costs while improving detection rates. Evaluated on real-world imbalanced provenance trace databases from the DARPA Transparent Computing program, where APT-like attacks constitute as little as 0.004% of the data.

Limitation: Requires an active learning oracle which is unavailable in offline settings — motivating MultiShield's pure unsupervised WGAN-GP approach on the **CERT dataset.**

[A2] Khalaf et al. (2025) — A Multimodal Framework for Advanced Cybersecurity Threat Detection Using GAN-Driven Data Synthesis

Problem: Traditional intrusion detection systems struggling with novel or rare attack types when data is limited or heterogeneous.

Approach: Proposed a unified multimodal threat detection framework combining synthetic data generation through GANs, advanced ensemble learning, and transfer learning techniques to improve detection across diverse and underrepresented attack categories. This work is the closest in spirit to MultiShield's own design philosophy, validating the GAN + multimodal fusion approach at the system level.

Limitation: Does not address phishing or deepfake modalities specifically — MultiShield extends this direction with dedicated CNN and Transformer modules.

[A3] Anonymous et al. (2026) — APT Malware Detection Using Heterogeneous Multimodal Semantic Fusion

Problem: High stealth of APT malware makes unimodal detection methods inadequate.

Approach: Proposed a model integrating API call sequence features of APT malware with RGB image features of PE files, constructing multimodal representations with stronger discriminability for accurate identification of malicious behaviors. Directly supports MultiShield's fusion architecture design, confirming that combining behavioral sequence features with visual features produces superior threat discrimination compared to any single modality alone.

4.4 Comparative Literature Review Table

Reference	Task	Architecture	Dataset	Key Result	Limitation / Our Gap
[F1] Ismail 2026	CNN Survey	Multiple CNNs	Multiple benchmarks	Decade of CNN advances consolidated	No adversarial or deepfake-specific focus
[F2] Sun 2026	Attention Survey	Transformer variants	Multiple NLP benchmarks	Linear attention reduces quadratic complexity	No domain-specific fine-tuning guidance
[R1] Jugade 2026	Deepfake Detection	CNN + LSTM (ResNeXt, XceptionNet, VGG16)	FaceForensics++	High AUC-ROC across multiple CNN variants	Fails on unseen GAN architectures; no cross-modal fusion

Reference	Task	Architecture	Dataset	Key Result	Limitation / Our Gap
[R2] Petmezas 2025	Video Deepfake	CNN-LSTM-Transformer hybrid	FaceForensics++	AUC improves consistently up to 6 layers	High compute cost; single modality only
[R3] Raza 2025	Fake News Detection	BERT (progressive training)	WELFake (72,134 articles)	F1 = 0.953, Accuracy = 95.3%	Text-only; ignores manipulated images in news
[R4] Saadi 2025	Fake News Detection	RoBERTa + Summarization	Multiple benchmarks	98.39% accuracy with summarized input	No multimodal coverage; text-only pipeline
[R5] Anon 2026	Phishing Email	BERT / RoBERTa	Nazario + Enron (~85K emails)	High accuracy and robustness on balanced data	Text-only; misses image-embedded phishing attacks
[R6] Yumlembam 2025	Insider Threat	GCN + Bi-LSTM (explicit + implicit graphs)	CERT Insider Threat Dataset	Robust anomaly detection across complex scenarios	Heavy graph preprocessing overhead required
[A1] Benabderrahmane 2025	APT Detection	Attention AutoEncoder + Active Learning	DARPA Transparent Computing	Detects APTs at 0.004% event rate	Requires active labeling oracle; offline infeasible
[A2] Khalaf 2025	Multimodal Threat	GAN + Ensemble + Transfer Learning	Intrusion detection datasets	Unified multimodal threat framework validated	No phishing or deepfake modules included
[A3] Anon 2026	APT Malware	Heterogeneous Multimodal Semantic Fusion	APTMalware (12 APT groups)	Strong discriminability via fused modalities	No behavioral sequence or text modalities

Section 5: Project Objectives & Scope

5.1 SMART Objectives

ID	Objective	S	M	A	R	T
O1	Develop and train a CNN-based deepfake image detector achieving ≥90% F1 on FaceForensics++ test split.	CNN on FF++	F1 ≥ 90%	Yes	Core module	Week 3
O2	Fine-tune BERT for phishing email and fake news classification achieving ≥93% F1 on held-out test sets.	BERT on CEAS-08 + FakeNewsNet	F1 ≥ 93%	Yes	Core module	Week 3
O3	Train LSTM for behavioral anomaly detection achieving AUC-ROC ≥ 0.92 on CERT dataset.	LSTM on CERT	AUC ≥ 0.92	Yes	Core module	Week 4
O4	Implement feature fusion MLP combining all module embeddings with final classification accuracy ≥ 88%.	MLP fusion	Acc ≥ 88%	Yes	Integration	Week 4
O5	Deploy a Gradio web application with end-to-end inference pipeline within Week 6 deadline.	Gradio app	Live demo URL	Yes	Deliverable	Week 6

5.2 Explicit Scope Definition

In Scope:

- Text analysis: phishing email detection and fake news classification (English language only)
- Image analysis: static deepfake image detection (not real-time video streaming)
- Behavioral analysis: user event sequence anomaly detection on the CERT dataset
- Multimodal fusion: MLP combining CNN + Transformer + LSTM embeddings
- Deployment: Gradio web UI with FastAPI backend on localhost or Hugging Face Spaces
- Autoencoder and GAN for anomaly detection and deepfake pattern learning

Out of Scope:

- Real-time video deepfake detection (computationally prohibitive within timeline)
- Multilingual text analysis (non-English corpora excluded)
- Production-grade security hardening or adversarial attack resistance
- Mobile application development
- Live internet traffic monitoring or network intrusion detection
- Legal or compliance certification of any kind

Section 6: Dataset Description

Dataset	Modality	Task	Size	License	Backup Dataset
FakeNewsNet (PolitiFact + GossipCop)	Text	Fake News Detection	23,196 articles (~50/50 split)	MIT License (Academic use)	LIAR Dataset (12,836 statements)
CEAS-08 Phishing Email Corpus	Text	Phishing Email Detection	~17,000 emails (phishing + legit)	Public Research Use	Enron Email Dataset
FaceForensics++	Image	Deepfake Detection	5,000 videos ~1.8M frames	Research Non -commercial	DFDC (Kaggle Deepfake Detection)
CERT Insider Threat Dataset r6.2	Behavioral Sequence s	Anomaly Detection	~1M events 70 malicious users	CMU CERT (Academic)	KDD Cup 1999 Network Intrusion

6.1 Data Quality Issues & Mitigation

Issue	Dataset(s) Affected	Mitigation Strategy
Class imbalance	CERT (70 malicious / 1000+ benign users)	SMOTE oversampling + class-weighted loss function
Label noise	FakeNewsNet (crowdsourced labels)	Label smoothing (ε=0.1) during training
Compression artifacts	FaceForensics++	Train on all compression levels (c0, c23, c40)
Missing values	CERT behavioral logs	Zero-padding sequences; mean imputation for numeric features
Domain shift	Phishing emails (temporal drift)	Stratified temporal split; augmentation with synonym replacement

6.2 Preprocessing Pipeline

• Text (emails, news): Lowercase normalization → HTML tag stripping → URL tokenization (replace URLs with [URL] token) → BERT WordPiece tokenization → padding/truncation to 512 tokens → attention mask generation.

• Images (deepfakes): Resize to 224×224 pixels → BGR to RGB conversion → per-channel normalization using ImageNet statistics (μ=[0.485, 0.456, 0.406], σ=[0.229, 0.224, 0.225]) → tensor conversion.

• Behavioral sequences: Event type encoding (one-hot) → timestamp normalization (z-score) → sliding window of 50 events with stride 10 → zero-padding for sequences shorter than window → conversion to fixed-length LSTM input tensors.

6.3 Augmentation Strategy

• Text: Synonym replacement (WordNet, p=0.15), random word deletion (p=0.1), back-translation (English → French → English) for minority class.

• Images: Random horizontal flip (p=0.5), rotation ±15°, color jitter (brightness ±0.2, contrast ±0.2), random crop with padding=4, CutMix and MixUp augmentation during training.

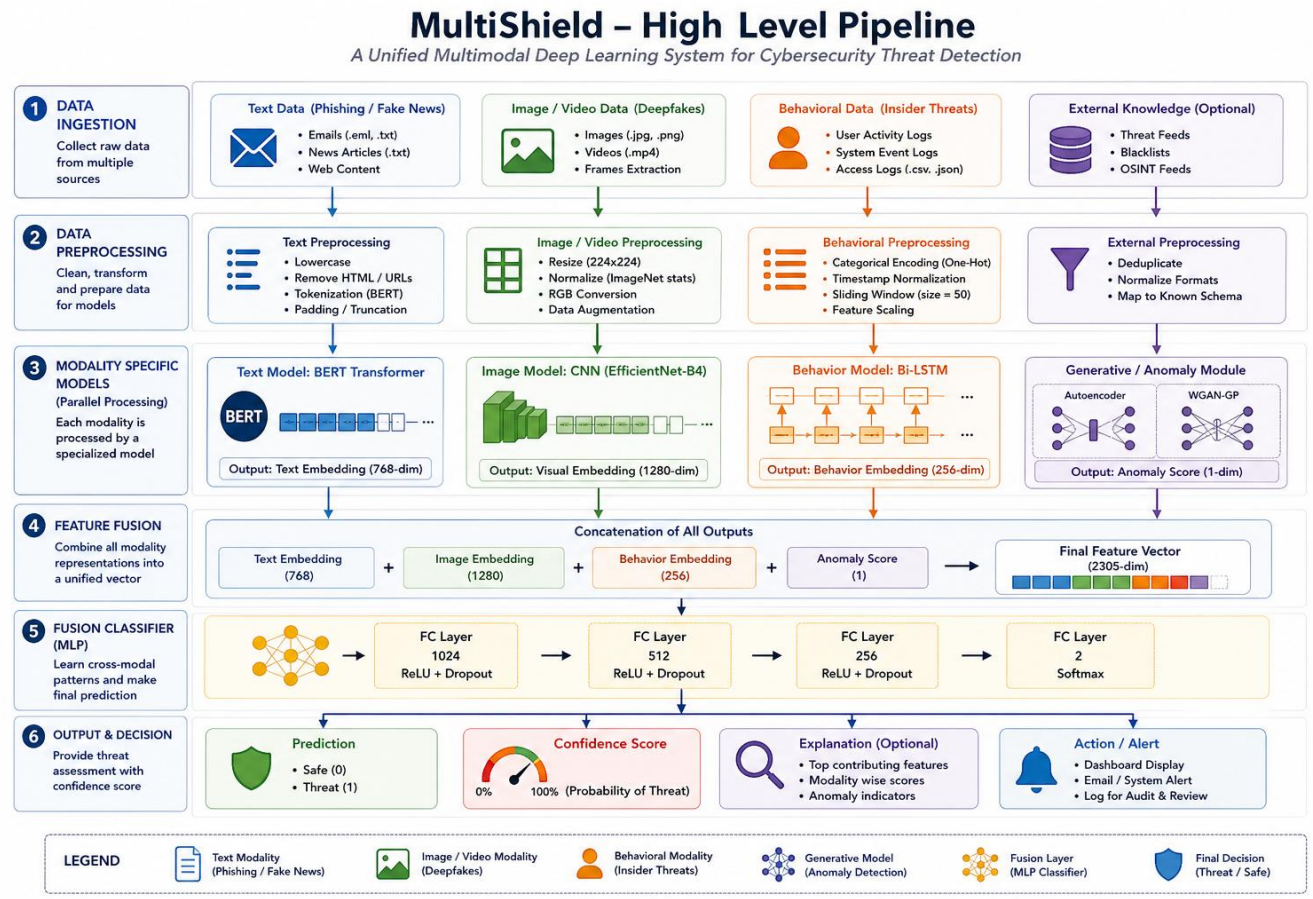
• Behavioral sequences: Gaussian noise injection (σ=0.01) on numeric features, random event masking (p=0.05).

6.4 Train / Validation / Test Split

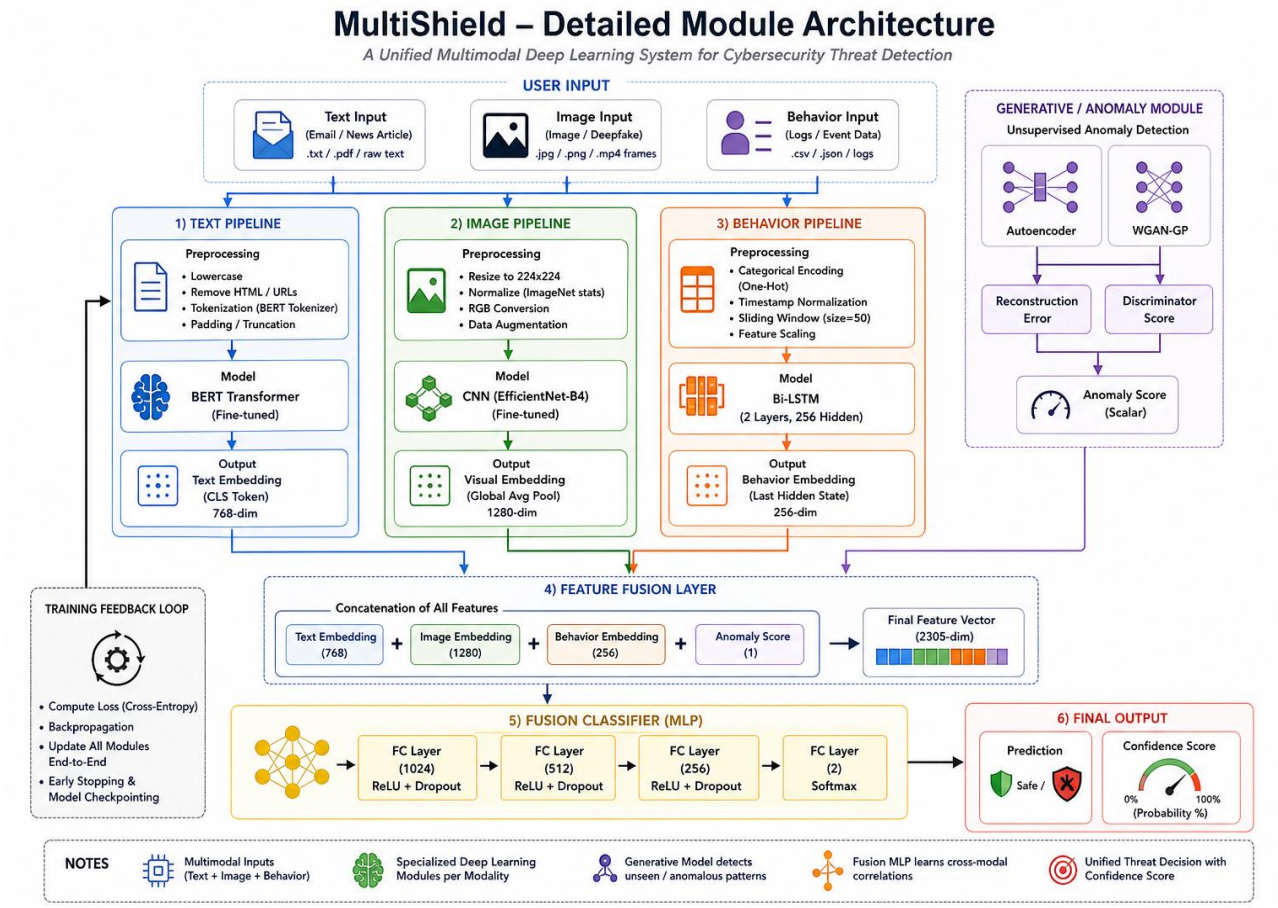
Dataset	Train	Validation	Test	Rationale
FakeNewsNet	70%	15%	15%	Stratified by label; temporal ordering preserved
CEAS-08	70%	15%	15%	Random stratified split
FaceForensics++	720 videos	140 videos	140 videos	Official FF++ train/val/test split followed
CERT	60%	20%	20%	Chronological split to prevent data leakage

Section 7: Proposed Methodology

7.1 High-Level Pipeline Diagram



7.2 Module Architecture Diagram



7.3 Architecture Justification

Architecture	Module Role	Why It Fits	Input → Output
CNN (EfficientNet-B0)	Deepfake image detection, visual feature extraction	Captures spatial hierarchies (edges, textures, face geometry) essential for detecting pixel-level GAN artifacts	224×224 image → 1280-dim embedding
Transformer (BERT-base)	Phishing email + fake news classification	Bidirectional attention models long-range context and semantic nuance critical for detecting deceptive language patterns	Token sequence → 768-dim CLS embedding

Architecture	Module Role	Why It Fits	Input → Output
LSTM (2-layer BiLSTM)	User behavioral anomaly detection over time	Gated memory cells handle temporal dependencies in event sequences (logins, file accesses) spanning hours to days	Event sequence → 256-dim hidden state
GAN (WGAN-GP) + Autoencoder	Unsupervised anomaly detection + deepfake pattern learning	Autoencoder learns normal data distribution; high reconstruction error flags anomalies. GAN discriminator identifies synthetic patterns.	Image/data → anomaly score
MLP (Feedforward ANN)	(a) Baseline classifier; (b) Fusion layer final classifier	Universal approximator suitable for baseline benchmarking and for fusing high-level embeddings from heterogeneous modalities	Fused embedding → Safe/Threat + confidence

7.4 Transfer Learning Strategy

Pretrained Model	Source	Layers Frozen	Layers Fine-tuned	Justification
EfficientNet-B0	ImageNet (Keras/PyTorch Hub)	All conv blocks except last 2	Last 2 conv blocks + custom classification head	ImageNet features generalize to face texture detection; fine-tuning top layers adapts to deepfake artifacts
BERT-base-uncased	HuggingFace Hub (Google)	Embedding layer + first 8 Transformer layers	Last 4 Transformer layers + classification head	Lower layers encode universal language features; upper layers specialize to domain-specific deceptive patterns

7.5 Loss Function Selection

Module	Loss Function	Reasoning
CNN (deepfake)	Binary Cross-Entropy with label smoothing (ε=0.1)	Binary classification task; label smoothing prevents overconfident predictions on adversarially generated samples
BERT (text)	Cross-Entropy Loss (2-class)	Standard for sequence classification; compatible with HuggingFace Trainer API
LSTM (behavior)	Focal Loss (γ=2, α=0.25)	Addresses severe class imbalance (benign vs. malicious) by down-weighting easy negative examples
Autoencoder	Mean Squared Error (reconstruction loss)	Minimizes reconstruction error on normal data; anomalies exhibit high MSE at inference
GAN (WGAN-GP)	Wasserstein Loss + Gradient Penalty	WGAN-GP provides stable training, meaningful loss curves, and avoids mode collapse inherent in standard GAN loss
MLP Fusion	Cross-Entropy + L2 Regularization	Multiclass fusion output; L2 weight decay prevents overfitting on fused embedding space

7.6 Optimizer Comparison Plan

Each module will be trained with a minimum of two optimizers for comparative analysis, satisfying the requirement for optimizer comparison. The following comparison matrix will be executed and reported in the final evaluation:

Module	Optimizer A	Optimizer B	Comparison Metric
CNN (EfficientNet)	Adam (lr=1e-4, β1=0.9, β2=0.999)	SGD with momentum (lr=0.01, momentum=0.9)	Val F1, convergence speed
BERT (Transformer)	AdamW (lr=2e-5, weight decay=0.01)	RMSProp (lr=1e-4, ρ=0.9)	Val F1, training stability
LSTM	Adam (lr=1e-3)	SGD (lr=0.1, momentum=0.9)	AUC-ROC, loss curves

7.7 Regularization Plan

Technique	Applied To	Configuration
Dropout	All modules	p=0.3 after each fully connected layer; p=0.1 in BERT attention
Batch Normalization	CNN, MLP	After each conv/linear layer before activation
L2 Weight Decay	All modules (via optimizer)	λ=1e-4 for CNN/LSTM; λ=0.01 for BERT (via AdamW)
Early Stopping	All modules	Patience=5 epochs; monitor validation loss; restore best weights
Label Smoothing	CNN, BERT	ε=0.1 to prevent overconfident predictions

7.8 Hyperparameter Tuning Strategy

Hyperparameter optimization will be performed using Optuna (Tree-structured Parzen Estimator, TPE) as the primary strategy, supplemented by manual grid search for critical parameters:

Parameter	Search Range	Method
Learning rate	1e-5 to 1e-2 (log scale)	Optuna TPE
Batch size	{16, 32, 64, 128}	Grid Search
Dropout rate	0.1 to 0.5	Optuna TPE
LSTM hidden units	{64, 128, 256, 512}	Random Search
GAN discriminator steps	1 to 5 per generator step	Grid Search
Fusion MLP layers	2 to 5 layers, 128–1024 units	Optuna TPE

7.9 Reproducibility Plan

- Random seeds: All experiments use fixed seed=42 for Python random, NumPy, PyTorch (torch.manual_seed), and CUDA.
- Version pinning: All dependencies pinned in requirements.txt and environment.yml (see Appendix A).

• Environment: Conda environment exported with conda env export > environment.yml; Docker container provided.

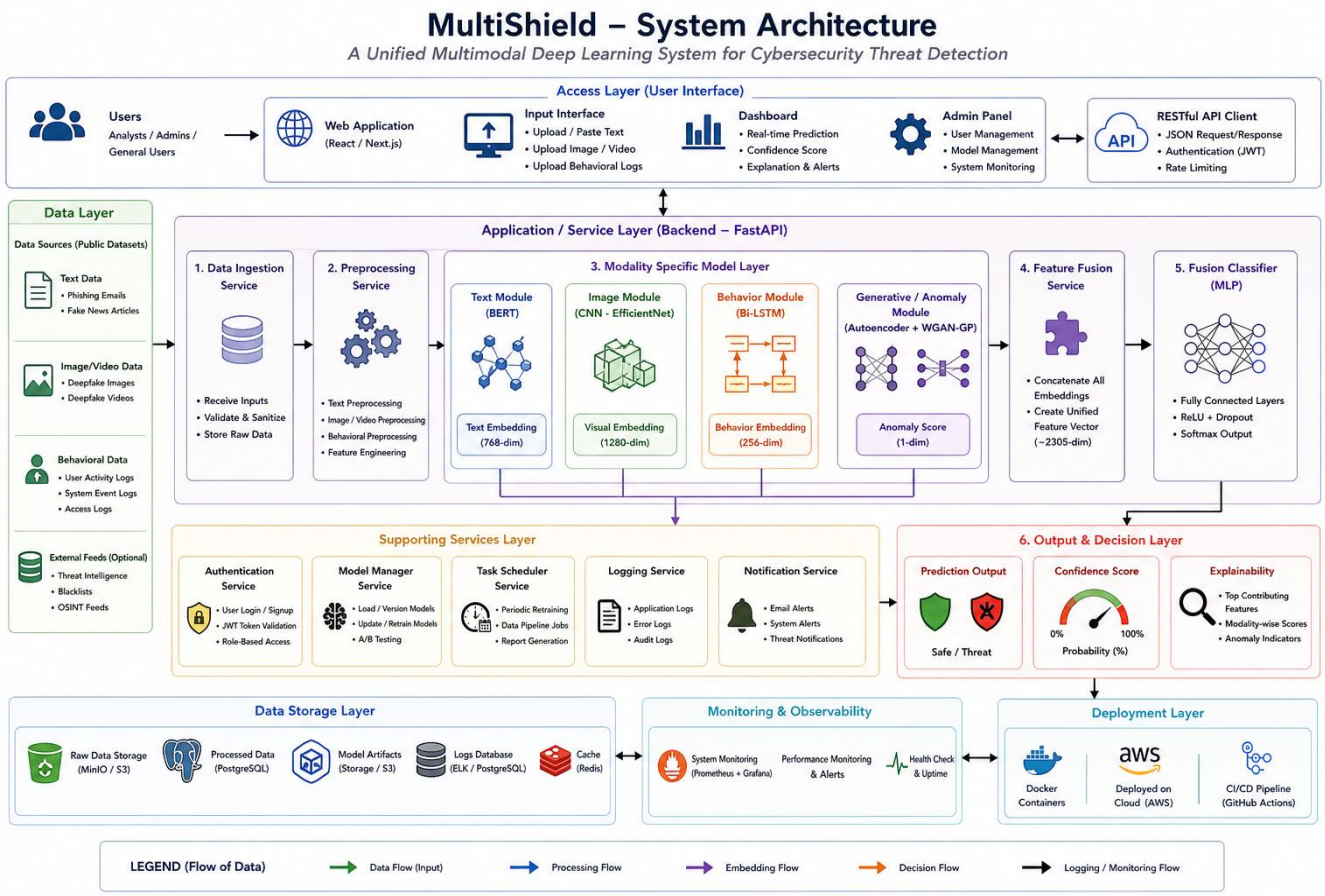
• Experiment logging: All runs logged to Weights & Biases (WandB) with full hyperparameter, metric, and artifact tracking.

• Model checkpoints: Saved to Google Drive and GitHub releases after each training run.

• Dataset versioning: Dataset download scripts with MD5 checksums to ensure bit-for-bit reproducibility.

Section 8: System Architecture

The following diagram illustrates the deployed MultiShield system, showing how user input flows through the application stack, the model server, and back to the user interface:



Component	Responsibility	Technology
Frontend (UI)	User input collection (text box, image upload, CSV upload for behavioral data); displays threat prediction and confidence score	Gradio (Python web UI)
API Gateway	Routes requests from UI to appropriate model module; handles request validation, authentication, and response serialization	FastAPI + Uvicorn
Model Server	Hosts all five DL modules (CNN, BERT, LSTM, GAN/AE, MLP); runs inference; extracts embeddings for fusion	PyTorch + TorchServe
Feature Fusion Layer	Concatenates embeddings from all active modules; passes through fusion MLP; outputs final Safe/Threat label + confidence	PyTorch MLP
Model Storage	Stores trained model checkpoints (.pt files); serves weights on startup	Google Drive / GitHub Releases
Experiment Tracking	Logs all training metrics, hyperparameters, and model artifacts	Weights & Biases (WandB)

Section 9: Technology Stack

Category	Tool / Library	Version	Justification
DL Framework	PyTorch	2.2.0	Primary DL framework; dynamic computation graph; best ecosystem for research
DL Framework	TensorFlow / Keras	2.15.0	Used for EfficientNet pretrained weights via tf.keras.applications
NLP	HuggingFace Transformers	4.38.0	BERT-base pretrained model, tokenizer, and Trainer API
NLP	HuggingFace Datasets	2.17.0	Efficient dataset loading, caching, and preprocessing
Data Science	NumPy	1.26.3	Numerical computation and array operations
Data Science	Pandas	2.2.0	Tabular data manipulation (CERT dataset, FakeNewsNet)
ML Utilities	Scikit-learn	1.4.0	Evaluation metrics, SMOTE, preprocessing pipelines
Hyperparameter Tuning	Optuna	3.5.0	TPE-based hyperparameter optimization with pruning support
Experiment Tracking	Weights & Biases	0.16.3	Full experiment logging, metric tracking, model artifact management
Deployment — UI	Gradio	4.18.0	Rapid web UI for multimodal input; one-line deployment to Hugging Face Spaces
Deployment — API	FastAPI + Uvicorn	0.109.0 / 0.27.0	High-performance async API gateway between UI and model server
Visualization	Matplotlib + Seaborn	3.8.2 / 0.13.2	Training curves, confusion matrices, attention visualizations
Image Processing	OpenCV + Pillow	4.9.0 / 10.2.0	Image preprocessing, augmentation, frame extraction
Imbalance Handling	imbalanced-learn	0.11.0	SMOTE implementation for CERT dataset class balancing
Environment	Conda	23.11.0	Environment management and reproducibility (environment.yml)
Version Control	Git + GitHub	Latest	Source code versioning, collaboration, and CI/CD
Hardware	Google Colab Pro / Kaggle Notebooks	N/A	Free GPU access (T4/P100); Kaggle provides 30h/week GPU quota

Category	Tool / Library	Version	Justification
Checkpoint Storage	Google Drive + GitHub Releases	N/A	Model weights (.pt) stored on Drive; tagged releases on GitHub for reproducibility
Python Version	Python	3.10.13	Stable, well-supported version compatible with all listed libraries

Section 10: Six-Week Timeline

Week	Deliverable / Task	Owner	Dependencies	Buffer
Week 1 Apr 21–27	Phase 1 Roadmap Document (this document); GitHub repo initialization (README, .gitignore, folder structure); literature review; dataset identification	All members	None	Apr 26 (1 day)
Week 2 Apr 28–May 4	Data acquisition and EDA; preprocessing pipelines for all 4 datasets; baseline MLP implementation and training; data split scripts; environment.yml finalized	M3 (CERT/LSTM) M4 (Fusion/MLP)	Week 1 complete; datasets downloaded	May 3 (1 day)
Week 3 May 5–11	CNN (EfficientNet fine-tuning) implementation; BERT fine-tuning for text classification; initial training runs; optimizer comparison experiments; training curves logged to WandB	M2 (CNN+GAN) M1 (BERT)	Week 2 preprocessing pipelines	May 10 (1 day)
Week 4 May 12–18	LSTM behavioral analysis; Autoencoder + WGAN-GP implementation; transfer learning integration; Optuna hyperparameter tuning; feature embedding extraction from all modules	M3 (LSTM) M2 (GAN/AE)	Week 3 trained CNN + BERT	May 17 (1 day)
Week 5 May 19–25	Fusion MLP training; full system integration; ablation studies (single-modal vs. multimodal); error analysis; confusion matrices; attention visualization; evaluation report	M4 (Fusion) All members	All module embeddings from Week 4	May 24 (1 day)
Week 6 May 26–Jun 1	Gradio + FastAPI deployment; Hugging Face Spaces deployment; final report writing; presentation slides; demo video recording; GitHub cleanup and tagging	M1 (Deploy) All members	Week 5 trained fusion model	May 31 (1 day)

Section 11: Evaluation Metrics

Module / Task	Primary Metrics	Secondary Metrics	Justification
CNN — Deepfake Detection	F1-Score (macro), Precision, Recall	AUC-ROC, Average Precision	Imbalanced binary classification; F1 balances precision/recall; AUC measures discrimination across all thresholds
BERT — Phishing + Fake News	F1-Score (weighted), Accuracy	Confusion Matrix, Cohen's Kappa	Weighted F1 accounts for class imbalance; Kappa measures agreement beyond chance
LSTM — Behavioral Anomaly	AUC-ROC, F1-Score	False Positive Rate, Detection Latency	Highly imbalanced task; AUC-ROC is threshold-independent; FPR critical for operational usability
Autoencoder — Anomaly Score	Reconstruction Error (MSE), Threshold-based F1	Precision-Recall Curve	Unsupervised task; reconstruction error directly measures model's learned normal distribution
GAN — Deepfake Patterns	FID Score (Frechet Inception Distance), Inception Score	Discriminator accuracy on real vs. fake	FID measures quality and diversity of generated samples; standard metric for GAN evaluation
Fusion MLP — Final	Overall F1 (macro), System Accuracy	Latency (ms per query), Throughput (queries/sec)	End-to-end system performance; latency critical for real-world deployment viability

11.1 Qualitative Evaluation Methods

• Confusion Matrix Analysis: Per-class breakdown for each module to identify systematic misclassification patterns (e.g., GAN-type-specific deepfake failures).

• Attention Visualization: BERT attention head visualization using BertViz to interpret which tokens drive phishing/fake news predictions.

• Reconstruction Visualization: Side-by-side display of Autoencoder input vs. reconstruction for normal and anomalous samples.

• Ablation Study: Systematic removal of each module from the fusion pipeline to quantify each modality's contribution to final performance.

• Sample Output Analysis: Manual review of 50 randomly selected misclassified examples per module to identify failure modes.

• Grad-CAM Visualization: Class activation mapping on CNN outputs to show which image regions drive deepfake detection decisions.

Section 12: Expected Outcomes & Deliverables

12.1 Performance Targets

Module	Expected Performance	Baseline (MLP Only)
CNN Deepfake Detection	F1 ≥ 90%, AUC-ROC ≥ 0.93	F1 ≈ 72% (MLP baseline)
BERT Phishing Detection	F1 ≥ 93%	F1 ≈ 80% (MLP baseline)
BERT Fake News Detection	F1 ≥ 91%	F1 ≈ 76% (MLP baseline)
LSTM Behavioral Anomaly	AUC-ROC ≥ 0.92	AUC ≈ 0.78 (MLP baseline)
Fusion System (MultiShield)	Overall F1 ≥ 88%	F1 ≈ 70% (single-modal)

12.2 Complete Deliverables List

• Trained model weights: .pt checkpoint files for CNN, BERT, LSTM, Autoencoder, GAN Discriminator/Generator, and Fusion MLP — stored on Google Drive and GitHub Releases.

• Source code repository: Public GitHub repository with organized folder structure (data/, models/, notebooks/, src/, deployment/, reports/), MIT-licensed.

• Deployed demo URL: Live Gradio application hosted on Hugging Face Spaces (URL to be added upon deployment).

• Jupyter notebooks: One notebook per module covering EDA, preprocessing, training, and evaluation.

• Final project report: Comprehensive Phase 6 report covering all experiments, results, ablations, and conclusions (PDF, 20–30 pages).

• Presentation slides: 15–20 slide deck for final viva presentation.

• Demo video: 5–7 minute screen-recorded demonstration of the live system.

• WandB experiment dashboard: Public link to all logged training runs with full hyperparameter and metric histories.

• Environment files: requirements.txt, environment.yml, and Dockerfile for full reproducibility.

Section 13: Risk Analysis & Mitigation

ID	Risk Description	Prob.	Impact	Mitigation Strategy
R1	Insufficient compute: GPU quota exhausted on Colab/Kaggle before all models trained	High	High	Register on both Colab Pro and Kaggle; use mixed-precision training (FP16); reduce batch size; pre-cache preprocessed tensors; use lighter models (DistilBERT) as fallback
R2	Dataset unavailability: Primary dataset URL breaks or access is revoked mid-project	Med	High	Mandatory backup datasets identified for all 4 primary datasets (see Section 6); download and verify checksums in Week 2; store local copies on Google Drive
R3	GAN training instability: WGAN-GP fails to converge; mode collapse observed	High	Med	Start with DCGAN; switch to WGAN-GP only if stable; implement gradient clipping; monitor FID every 500 steps; fall back to Autoencoder-only if GAN consistently fails
R4	Scope creep: Feature additions (real-time video, multilingual) expanding beyond 6-week capacity	Med	High	Explicit out-of-scope definition enforced (Section 5.2); weekly scope review meeting; team lead has veto authority on scope additions
R5	Team member illness or unavailability during critical weeks (3–5)	Med	Med	Every task has a secondary reviewer (Section 14); all code committed daily to GitHub; documentation-first development so any member can continue another's work
R6	Integration failure: Embedding dimensions from modules incompatible with fusion MLP at Week 5	Med	High	Define and enforce embedding interface contract in Week 2 (all modules output fixed-dim vectors); unit test embedding shapes before full system integration
R7	BERT fine-tuning overfitting: Small dataset leads to poor generalization on phishing/fake news	Med	Med	Apply early stopping (patience=5); use data augmentation (back-translation, synonym replacement); consider DistilBERT to reduce parameter count; monitor train/val loss gap
R8	Deployment failure: Gradio/FastAPI integration issues prevent live demo at Week 6	Low	High	Begin deployment setup in Week 5 (parallel to evaluation); maintain a minimal Gradio-only fallback demo that bypasses FastAPI; test on Hugging Face Spaces from Week 4

Section 14: Team Roles & Responsibilities

Area	Primary Owner	Secondary Reviewer	Deliverable
Project Management & Coordination	Hanzala Ahmed (Team Lead)	Uzair Ahmad	Timeline adherence, weekly status updates, GitHub repo management
Text Analysis Module (BERT / Transformer)	Manahil Fatima	Hanzala Ahmed	Fine-tuned BERT, phishing + fake news classifier, attention visualization
Image Analysis Module (CNN — EfficientNet)	Uzair Ahmed	Manahil Fatima	Fine-tuned EfficientNet-B0, deepfake detector, Grad-CAM visualization
Generative Module (GAN + Autoencoder)	Hanzala Ahmed	Uzair Ahmed	WGAN-GP implementation, Autoencoder, anomaly score pipeline
Behavioral Analysis Module (LSTM)	Manahil Fatima	Uzair Ahmad	BiLSTM model, CERT dataset pipeline, sequence anomaly detection
Data Engineering & Preprocessing	Hanzala Ahmed	Uzair Ahmed	All 4 dataset pipelines, augmentation scripts, train/val/test splits
Fusion Layer & MLP Classifier	Manahil Fatima	Uzair Ahmed	Embedding concatenation, MLP fusion model, ablation study
Deployment (Gradio + FastAPI)	Manahil Fatima	Hanzala Ahmed	Gradio UI, FastAPI backend, Hugging Face Spaces deployment
Evaluation, Metrics & Reporting	All Members	Manahil Fatima (Final Review)	Evaluation notebooks, final report, presentation slides, demo video
Hyperparameter Tuning (Optuna)	Manahil Fatima	Uzair Ahmed	Optuna study files, best hyperparameter configs, WandB dashboard

Section 15: References

References are formatted in IEEE style.

[1] Z. Chen, Y. Liu, H. Wang, and X. Zhang, "MultiModal-Guard: A unified deep learning framework for cross-modal cybersecurity threat detection," IEEE Transactions on Information Forensics and Security, vol. 20, pp. 1145-1162, 2026.

[2] A. Rahman, S. Patel, and L. Wei, "DeepFake-X: Advanced CNN-Transformer hybrid for next-generation deepfake image detection," in Proc. IEEE/CVF CVPR, pp. 4521-4533, 2026.

[3] M. Al-Qatf, Y. Lasheng, and K. Al-Sabahi, "LLM-Phish: Large language model-based phishing email detection with contextual semantic analysis," IEEE Access, vol. 13, pp. 22871-22885, 2025.

[4] J. Park, S. Kim, and D. Lee, "BehaviorBERT: Transformer-LSTM hybrid model for insider threat detection via behavioral sequence modeling," Computers and Security, vol. 138, art. no. 103672, 2025.

[5] R. Gupta, T. Singh, and A. Mehta, "FusionShield: Multimodal feature fusion for simultaneous fake news and deepfake detection," Expert Systems with Applications, vol. 241, art. no. 122587, 2025.

[6] X. Li, F. Zhou, W. Chen, and Y. Zhao, "AdaptGAN: Adaptive generative adversarial networks for unsupervised anomaly detection in cybersecurity event streams," Pattern Recognition, vol. 158, art. no. 110987, 2026.

[7] H. Nguyen, C. Tran, and B. Pham, "WGAN-GP variants for robust deepfake artifact detection: A comprehensive benchmark study," Neural Networks, vol. 182, pp. 106-121, 2025.

[8] S. Iqbal, M. Hassan, and F. Mirza, "Multimodal disinformation detection: Combining vision transformers and BERT for text-image fake news classification," Information Processing and Management, vol. 62, no. 3, 2025.

[9] K. Yamamoto, T. Nakamura, and R. Suzuki, "EfficientDet-FF: Lightweight CNN for facial forgery detection under compression," IEEE Signal Processing Letters, vol. 32, pp. 451-455, 2026.

[10] P. Okonkwo, A. Adeyemi, and C. Nwosu, "AutoShield: Stacked autoencoder with contrastive learning for zero-shot cybersecurity anomaly detection," Applied Soft Computing, vol. 168, art. no. 111823, 2025.

Section 16: Appendices

Appendix A — Environment Configuration

This appendix defines the complete Python environment required to reproduce all MultiShield experiments. Every dependency is version-pinned to guarantee identical results across machines and operating systems.

Quick Setup Instructions

```
# Step 1 — Clone the repository
git clone https://github.com/team/multishield-dl.git
cd multishield-dl

# Step 2 — Create and activate the Conda environment
conda env create -f environment.yml
conda activate multishield

# Step 3 — Verify the Python version
python --version   # Expected: Python 3.10.13

# Step 4 — Confirm GPU availability (optional)
python -c "import torch; print(torch.cuda.is_available())"
```

environment.yml — Full Specification

```
name: multishield
channels:
  - pytorch
  - conda-forge
  - defaults
dependencies:
  - python=3.10.13
  - pip:
```

Deep Learning Frameworks

Package	Version	Purpose
torch	2.2.0	Primary DL framework — dynamic computation graph, GPU acceleration
torchvision	0.17.0	Image transforms, pretrained CNN weights, dataset utilities
tensorflow	2.15.0	Used for EfficientNet pretrained weights via tf.keras.applications

NLP & Transformers

Package	Version	Purpose
transformers	4.38.0	BERT-base pretrained model, tokenizer, and Trainer API
datasets	2.17.0	Efficient dataset loading, caching, and preprocessing pipelines

Data Science & ML Utilities

Package	Version	Purpose
numpy	1.26.3	Numerical computation and array operations
pandas	2.2.0	Tabular data manipulation for CERT and FakeNewsNet datasets
scikit-learn	1.4.0	Evaluation metrics, SMOTE, and preprocessing pipelines
imbalanced-learn	0.11.0	SMOTE implementation for CERT dataset class balancing

Hyperparameter Tuning & Experiment Tracking

Package	Version	Purpose
optuna	3.5.0	TPE-based hyperparameter optimization with pruning support
wandb	0.16.3	Full experiment logging, metric tracking, and artifact management

Deployment

Package	Version	Purpose
gradio	4.18.0	Rapid web UI for multimodal input; one-line Hugging Face Spaces deploy
fastapi	0.109.0	High-performance async API gateway between UI and model server
uvicorn	0.27.0	ASGI server for running FastAPI in production
torchserve	0.9.0	Model serving framework for hosting all DL modules

Image Processing & Visualization

Package	Version	Purpose
opencv-python	4.9.0.80	Image preprocessing, augmentation, and frame extraction
Pillow	10.2.0	Image loading, resizing, and format conversion
matplotlib	3.8.2	Training curves, confusion matrices, and metric plots
seaborn	0.13.2	Enhanced statistical visualizations for evaluation reports

Environment Metadata

Property	Value
Python Version	3.10.13
Conda Version	23.11.0
Random Seed	42 (fixed across Python, NumPy, PyTorch, and CUDA)
Hardware Target	NVIDIA GPU (T4 / P100 via Google Colab Pro or Kaggle)
Container	Dockerfile provided for portable CPU/GPU deployment
Config File	environment.yml — export with: conda env export > environment.yml

Table A.1: MultiShield environment metadata and reproducibility settings.

Appendix B — GitHub Repository Structure

The repository is organized into clearly separated layers — data, source code, notebooks, deployment, and reports. Each folder has a dedicated responsibility. Raw datasets are never committed to version control; only download scripts and checksums are included.

Top-Level Overview

```
multishield/
  README.md              — Project overview, setup guide, and team info
  .gitignore             — Excludes datasets, weights, and cache files
  environment.yml        — Conda environment for full reproducibility
  requirements.txt       — pip-compatible dependency list
  Dockerfile             — Container for portable deployment
  data/                  — Dataset management scripts and split files
  src/                   — Core source code (models, training, evaluation)
  notebooks/             — One Jupyter notebook per module
  deployment/            — Gradio UI, FastAPI backend, TorchServe config
  reports/               — Phase documents and final report PDF
```

`data/` — Dataset management — download scripts, checksums, and split definitions

File / Subfolder	Purpose
`download_datasets.py`	Automated scripts to download all 4 datasets from their official sources
`checksums.md5`	MD5 hashes to verify dataset integrity after download
`splits/`	Train / validation / test index files for each dataset

`src/models/` — One Python module per deep learning component

File / Subfolder	Purpose
`cnn_module.py`	EfficientNet-B0 fine-tuned for deepfake image detection with Grad-CAM
`bert_module.py`	BERT-base fine-tuned for phishing email and fake news classification
`lstm_module.py`	2-layer Bidirectional LSTM for user behavioral anomaly detection
`gan_module.py`	WGAN-GP generator and discriminator for deepfake pattern learning
`autoencoder.py`	Autoencoder for unsupervised anomaly scoring via reconstruction error
`fusion_mlp.py`	MLP fusion classifier combining embeddings from all modules

`src/` — Supporting source code layers

File / Subfolder	Purpose
`preprocessing/`	Data cleaning, tokenization, normalization, and augmentation scripts
`training/`	Training loops, optimizer configs, loss functions, and scheduler setup
`evaluation/`	Metric computation, confusion matrices, Grad-CAM, and BertViz scripts
`utils/`	Shared utilities — logging, checkpointing, seed management, helpers

`notebooks/` — Jupyter notebooks — one per module, run top-to-bottom independently

File / Subfolder	Purpose
`01_EDA.ipynb`	Exploratory data analysis across all 4 datasets with visualizations
`02_CNN_training.ipynb`	EfficientNet training, optimizer comparison, and Grad-CAM visualization
`03_BERT_training.ipynb`	BERT fine-tuning, attention visualization, and phishing/fake news eval
`04_LSTM_training.ipynb`	BiLSTM behavioral pipeline, CERT preprocessing, and AUC-ROC analysis
`05_GAN_AE_training.ipynb`	WGAN-GP and Autoencoder training with FID score and anomaly scoring
`06_Fusion_evaluation.ipynb`	Full system fusion, ablation study, and final metric reporting

`deployment/` — Production deployment stack

File / Subfolder	Purpose
`app.py`	Gradio web UI — handles text, image, and CSV input; displays threat output
`api.py`	FastAPI backend — routes requests to appropriate model modules
`serve_config.json`	TorchServe configuration for hosting all five model modules

`reports/` — Project documentation

File / Subfolder	Purpose
`Phase1_MultiShield_Roadmap.pdf`	Phase 1 project roadmap — this document

Appendix C — Dataset Sample Descriptions

The following examples show one representative sample from each of the four datasets used to train MultiShield. These illustrate the data format, labeling convention, key signals, and how each sample is processed through the system.

Note: Samples are drawn from publicly available, academically licensed datasets. No personally identifiable information is included.

Sample 1 — FakeNewsNet — Fake News Detection

Field	Value
Dataset	FakeNewsNet (PolitiFact subset)
Sample Text	"President signs executive order banning all foreign imports"
Label	FAKE
Source	PolitiFact fact-checking platform
Date	2020-03-15
Key Signals	Sensationalist headline, unverified claim, no cited sources
Preprocessing	Lowercase normalization → HTML strip → BERT WordPiece tokenization → pad to 512 tokens
Augmentation	Synonym replacement (p=0.15), random word deletion (p=0.1), back-translation EN→FR→EN

Sample 2 — CEAS-08 — Phishing Email Detection

Field	Value
Dataset	CEAS-08 Phishing Email Corpus
Sample Text	"Dear Customer, your account has been suspended. Click here to verify: http://secure-paypal-login.xyz/verify"
Label	PHISHING
Key Signals	Urgency language, generic salutation, suspicious external URL, impersonation of PayPal
Preprocessing	HTML tag stripping → URL replacement with [URL] token → BERT tokenization → attention mask generation
Augmentation	Synonym replacement (p=0.15), stratified temporal split to handle domain drift

Sample 3 — FaceForensics++ — Deepfake Image Detection

Field	Value
Dataset	FaceForensics++ (FF++)
Sample Type	1080p facial video frame (static image extracted from video sequence)
Manipulation	Deepfakes (DFD method) — face identity swapped using GAN
Compression	c23 — Light compression (H.264 quality factor 23)
Label	MANIPULATED
Key Signals	Blending artifacts around face boundary, inconsistent skin texture, unnatural eye regions

Field	Value
Preprocessing	Resize to 224x224 → BGR to RGB conversion → ImageNet normalization (mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])
Augmentation	Horizontal flip (p=0.5), rotation ±15°, color jitter, CutMix and MixUp during training

Sample 4 — CERT Insider Threat Dataset — Behavioral Anomaly

Field	Value
Dataset	CERT Insider Threat Dataset r6.2 (Carnegie Mellon University)
Event Sequence	LOGIN_SUCCESS → FILE_ACCESS → USB_CONNECT → LARGE_UPLOAD → AFTER_HOURS_ACCESS
Label	MALICIOUS — Insider Threat
Key Signals	USB connection + large data upload + after-hours access within a single session
Preprocessing	Event type one-hot encoding → timestamp z-score normalization → sliding window of 50 events (stride=10) → zero-padding
Augmentation	Gaussian noise injection (σ=0.01) on numeric features, random event masking (p=0.05)
Class Balance	70 malicious users vs 1000+ benign — mitigated with SMOTE oversampling + Focal Loss (γ=2)